Lab guide

Configure the Computer identity recovery demo

Step-by-step lab guide for DevRev sales engineers.

This document shows how to stand up the **Keycloak password reset** demo that is already running in org **dcm-test**. When you finish, Computer can check, unlock, and reset a Keycloak account in chat, and a ticket can do the same with slash commands.

Use this guide in a lab or the night before a meeting. In the meeting, use the companion talk track: se-demo-script.pdf (PDF). A print PDF of this guide is se-configuration-guide.pdf.

Follow DevRev language guidelines when you write or present this demo: knowledgeable, approachable, and forward-thinking. Use American English and sentence case. Say **Computer**, **DevRev**, and **PLuG** exactly that way. Computer is the AI teammate that acts inside guardrails, not a chatbot.

What you are building

Computer is the AI teammate that remembers the business and **acts** inside guardrails. This demo is a service desk story: an employee is locked out of SSO, they ask Computer in plain language, and Computer calls Keycloak through org skills. Credentials never enter the chat.

```
Employee  →  Computer chat
                 │
                 ├─ KeycloakCheckAccount   (read status)
                 ├─ KeycloakUnlockAccount  (clear brute-force lockout)
                 └─ ResetPassword          (send UPDATE_PASSWORD email)
                        │
                        ▼
             HTTPS ngrok tunnel
                        │
                        ▼
             Laptop Keycloak (realm account-unlock)
```

The **Keycloak Password Reset** snap-in is the same recovery path on a ticket: `/check_account`, `/unlock_account`, `/reset_password`. Use Computer for the live story. Use the ticket when you want to show work-item automation.

Time and roles

Step	Who	Typical time
Keycloak + realm import	SE laptop	10 minutes

Step	Who	Typical time
ngrok tunnel	SE laptop	5 minutes
Snap-in connection	DevRev org admin	10 minutes
Computer skills (already in dcm-test)	SE / solutions	15 minutes the first time
Dry run	SE	10 minutes

Reuse **dcm-test** when you can. Recreate the skills only if you are building a fresh org.

Prerequisites

- Docker Desktop (or compatible) on the demo laptop
- An ngrok account and authtoken
- A DevRev org where you can install snap-ins and edit Computer skills (example: dcm-test)
- This repository, branch with `keycloak-password-reset/`
- Optional: DevRev CLI (`devrev`) if you will publish a new snap-in version

Do not commit the ngrok URL, authtoken, client secret, or a personal access token.

Reference environment (dcm-test)

These objects already exist in **dcm-test**. Confirm them before you rebuild anything.

Object	Reference
Org	https://app.devrev.ai/dcm-test/
Snap-in	Keycloak Password Reset, bot **Keycloak Password Reset Bot** (SVCACC-69)
Computer	`ai_agent/4`, slug `computer`
Password Reset Assistant	`ai_agent/6`
Skills	`KeycloakCheckAccount` (workflow 35), `KeycloakUnlockAccount` (36), `ResetPassword` (33)
Published skill versions	**33.8 / 35.8 / 36.8** (or later)
Realm	`account-unlock`
Client	`unlock-agent` (confidential, service account)

Current public Keycloak origin (changes if you restart a free ngrok tunnel):

```
https://urologist-supernova-effective.ngrok-free.dev/
```

Always include the trailing slash when you paste this into DevRev.

Part 1. Start Keycloak

From the repository root:

```
docker compose -f keycloak-password-reset/docker-compose.yml up
```

Wait until the container is healthy. Admin console on the laptop:

- URL: `http://localhost:8080`
- User: `admin`
- Password: `admin`

The compose file imports `keycloak-password-reset/realm/account-unlock-realm.json`. That realm turns on brute-force protection and creates:

Username	Email	Password
`demo.user`	`demo.user@example.com`	`DemoPass123!`
`danielcarvajal`	`daniel.carvajal@devrev.com`	`DemoPass123!`

Live **dcm-test** Keycloak also has `testuser` / `testuser@yourcompany.com`. Create that user in the admin console if your import does not include it.

The confidential client `unlock-agent` must have a service account with realm-management roles `manage-users`, `view-users`, and `query-users`. The imported realm already grants those roles.

Do **not** set `KC_HOSTNAME` to a free, ephemeral ngrok URL. That pins Keycloak to a dead host the next time the tunnel changes.

Brute-force lockout must stay locked until the API

Keycloak 26 has three brute-force **modes**. The imported realm used **Lockout temporarily**: after three failed passwords the user stayed `enabled: true`, and Keycloak cleared the lockout after **Wait Increment** (60 seconds). That looks like the account unlocked itself.

For this demo, use **Lockout permanently**. Keycloak then sets `enabled: false` after three failures. The account stays disabled until an administrator or the snap-in / Computer skill re-enables it.

In the admin console (realm `account-unlock`):

1. **Realm settings → Security defenses → Brute force detection**
2. Set **Brute force mode** to **Lockout permanently**
3. Set **Max login failures** to `3`
4. Leave **Quick login check milliseconds** at `1000` and **Minimum quick login wait** at `1 minute` (or raise the check so rapid demo clicks do not trigger the short quick-login wait)

Do **not** choose **Lockout temporarily**. Do **not** choose **Lockout permanently after temporary lockout** unless you want one or more time-limited lockouts first.

A running container does not re-import the realm. Change the live realm in the console, or recreate the volume after updating `realm/account-unlock-realm.json` (`permanentLockout` is `true`).

Unlock is two Admin API calls. The snap-in already does both:

1. `DELETE /admin/realms/account-unlock/attack-detection/brute-force/users/{id}`
2. `PUT /admin/realms/account-unlock/users/{id}` with `enabled: true`

A Computer skill that only deletes lockout will leave the user disabled. The Password Reset snap-in `/unlock_account` path re-enables the user.

Wait more than one second between failed logins when you stage the demo. Failures faster than **Quick login check milliseconds** use the temporary **Minimum quick login wait**, even in permanent mode, until the failure count reaches three.

Part 2. Expose Keycloak to DevRev

DevRev-hosted skills and snap-in functions cannot call `localhost`. Tunnel port 8080:

```
ngrok config add-authtoken <your-authtoken>
ngrok http 8080
```

Copy the **https** origin, for example `https://urologist-supernova-effective.ngrok-free.dev/`.

Prove the tunnel reaches Keycloak (do not judge this by opening `/admin` in Chrome on a free ngrok URL):

```
curl -sS -H 'ngrok-skip-browser-warning: true' \
  https://<your-subdomain>.ngrok-free.dev/realms/account-unlock/.well-known/openid-configuration
```

You want JSON with `issuer` and `token_endpoint`, not the ngrok HTML warning.

Leave Docker and ngrok running for the whole meeting. If the laptop sleeps, the demo dies.

Part 3. Install the snap-in

1. In the DevRev org, open **Settings → Snap-ins**.
2. Install **Keycloak Password Reset** (or create a version from this repo):

```
bash devrev profiles authenticate -o <org-slug> -u <you@example.com> devrev snap_in_version create-one --path ./keycloak-password-reset --create-package devrev snap_in draft
```

3. Create a **Keycloak Admin** connection:

Field	Value
Keycloak URL	`https://<ngrok-host>/` (trailing slash)
Realm	`account-unlock`
Client ID	`unlock-agent`
Client secret	The live client secret from Keycloak, not a value from git

4. Set organization input **Keycloak URL** to the same https origin if you are not storing the URL inside the connection.
5. Deploy.

Deploy registers `/check_account`, `/unlock_account`, and `/reset_password` as the snap-in bot. If activate fails with Unauthorized on `command` objects, grant **Command Interactor** to **Keycloak Password Reset Bot**.

After a tunnel change, edit the connection, update **Keycloak URL**, and deploy again. Do not leave a stale host in the keyring.

Part 4. Attach Computer skills

Computer in **dcm-test** already has three workflow-backed skills. Each skill is an `ai_agent_skill_trigger` plus HTTP steps. The model only passes `email` and/or `username`. Method, URL, and the client secret stay on the workflow.

Skill	Workflow	What it does
`KeycloakCheckAccount`	35	Find user, read brute-force lockout
`KeycloakUnlockAccount`	36	Find user, `DELETE` lockout
`ResetPassword`	33	Find user, `PUT execute-actions-email` with `UPDATE_PASSWORD`

Keep **WebSearch** on Computer when you replace the skill list. A `skills.set` call replaces the entire list.

Each HTTP step must send:

- `ngrok-skip-browser-warning: true`
- `Authorization: Bearer <token>`

Get Token is `POST /realms/account-unlock/protocol/openid-connect/token` with client credentials. Transform the response with jq `.access_token`. Build the header with DevRev JSONata only:

```
"Bearer " & $replace($string($get('get_token','output').body), '"', '')
```

Do **not** use `$eval`. DevRev JSONata does not implement it, and Find User fails while building headers (`cannot call non-function $eval`).

Find User must search, not exact-email only:

```
GET /admin/realms/account-unlock/users?search=<identity>
```

Map `@devrev.ai` to `@devrev.com` in that search string. Strip hyphens from a username that has no `@` (`daniel-carvajal` → `danielcarvajal`).

After you publish a skill version, start a **new** Computer chat. An old session keeps the previous input schema.

Part 5. Identity mapping

DevRev login and Keycloak are not the same mailbox. Teach Computer (and yourself) this table.

What the employee says	What Keycloak has	How lookup works
"Unlock my account" (Daniel)	Username `danielcarvajal`, email `daniel.carvajal@devrev.com`	Try DevRev email, then `@devrev.com`, then username
`daniel.carvajal@devrev.ai`	`daniel.carvajal@devrev.com`	Alias `@devrev.ai` → `@devrev.com`
`danielcarvajal`	Same user	Username search
`daniel-carvajal`	`danielcarvajal`	Hyphens stripped
`testuser@yourcompany.com`	Username `testuser`	Exact email or search

A requester may manage **their own** account. Do not reset someone else unless they clearly name that person's email or username.

Part 6. Verify before you train or demo

Keycloak from the laptop

```
TOKEN=$(curl -sS -H 'ngrok-skip-browser-warning: true' \
  -d 'grant_type=client_credentials&client_id=unlock-agent&client_secret=<secret>' \
  https://<ngrok-host>/realms/account-unlock/protocol/openid-connect/token \
  | jq -r .access_token)

curl -sS -H "Authorization: Bearer $TOKEN" \
  -H 'ngrok-skip-browser-warning: true' \
  "https://<ngrok-host>/admin/realms/account-unlock/users?search=danielcarvajal"
```

You want HTTP 200 and a JSON array. Use `jq -r` so the header is not `Bearer "eyJ..."`.

Computer

1. Open Computer in dcm-test (search bar).
2. Start a new chat.
3. Type `check danielcarvajal`.
4. Confirm Computer reports enabled / lockout status without asking for a method, URL, or secret.

Ticket (optional)

On any ticket discussion:

```
/check_account danielcarvajal
/unlock_account danielcarvajal
```

`/reset_password … --temp` posts a temporary password as an **internal** comment. Never read that password on a customer-visible thread.

ngrok inspector

Healthy Computer run:

```
POST /realms/account-unlock/protocol/openid-connect/token             200
GET  /admin/realms/account-unlock/users?search=danielcarvajal        200
GET  /admin/realms/.../brute-force/users/<uuid>                      200
```

Broken header or empty identity:

```
POST /realms/.../token                                               200
GET  /admin/realms/account-unlock/users                              401
GET  /admin/realms/.../brute-force/users/                            401
```

Token 200 then Admin API 401 means the Bearer token is missing or quoted. A lockout path that ends at `/users/` means Find User did not return an id.

Part 7. Checklist before a customer meeting

1. Docker Keycloak is up.
2. ngrok still forwards to port 8080.
3. Curl against the current https origin returns JSON.
4. Snap-in **Keycloak URL** matches that origin.
5. Computer skill HTTP URLs match that origin (workflows 33 / 35 / 36).
6. You started a **new** Computer chat after the last skill publish.
7. You can lock `testuser` with three bad passwords if you want a live unlock.
8. You will not share the client secret, PAT, or raw JWT on the call.

If the ngrok host changed since the last meeting, update the snap-in connection **and** the three skill workflows before you join.

Troubleshooting

Symptom	Fix
Chrome `net::ERR_ABORTED` on `/admin`	Free ngrok interstitial. Verify with curl and the skip header. Do not pin `KC_HOSTNAME` to an ephemeral host.
Snap-in or Computer cannot reach Keycloak	Tunnel down, laptop asleep, or stale URL in the connection / skill.
Find User: `cannot call non-function $eval`	Header used `$eval`. Publish skills 33.8+ with `$replace` only.
Token 200, users 401	Quoted or missing `Authorization`. Confirm jq `.access_token` and the `$replace` header.
“No Keycloak user” for Daniel	You searched `@devrev.ai` only. Use alias or username `danielcarvajal`.
Reset email fails	Realm SMTP is not configured. Use `/reset_password <user> --temp` for the demo.
Lockout clears after ~60s and user stays enabled	Realm is **Lockout temporarily**. Set **Lockout permanently** (see Brute-force lockout).
Computer unlocks lockout but user is still disabled	Permanent lockout set `enabled: false`. The skill must `PUT` the user with `enabled: true`, not only `DELETE` the brute-force record.
Snap-in activate Unauthorized on commands	Grant **Command Interactor** to the snap-in bot.

Language for enablement

Use this vocabulary with other SEs and with customers.

Use	Avoid
Computer	“the chatbot”, “Devrev AI”, “ChatGPT for IT”
Computer skill	“random HTTP tool the model invents”
Service desk automation	“a cool hack we wired up”
Guardrails	“trust us, it is safe”
DevRev	Devrev, DEVREV, devrev
PLuG	plug, PLUG

Lead with the outcome: the employee gets back into work. Then show that Computer acted on a governed skill, not a pasted password.

When you are ready to run the meeting, open se-demo-script.pdf.
